

How do I check for a null entered with an input statement in python

The short answer: in **Python**, an “empty” input is always the empty string `""`, never `None`, so you check for it explicitly.

Below is the clean, deterministic way to handle it, with multiple patterns depending on what you want.

✅ The core rule

`input()` **never returns** `None`. If the user presses Enter without typing anything, Python returns:

Code

```
""
```

So your null-check is really an **empty-string check**.

🔧 Pattern 1 — Basic empty-input check

python

```
value = input("Enter something: ")
```

```
if value == "":
```

```
    print("No input provided")
```

```
else:
```

```
    print("You entered:", value)
```

How do

How do I enter a null in a string in python

The key point: **Python does not have a “null character”** for strings the way languages like C do. But you *can* insert:

- an **empty string** (`""`)
- a **null byte** (`"\0"`)
- or **None** (but that is *not* a string)

Here are the exact ways to do each, depending on what you mean by “null”.

❖ 1 — Insert an actual empty string (Python’s normal “null” for text)

This is the most common meaning.

python

```
s = ""
```

Or inside another string:

python

```
name = "Bob" + ""
```

This is Python’s equivalent of “no characters”.

❖ 2 — Insert a null byte (\0) inside a Python string

If you literally want the **null character** used in C strings:

python

```
s = "hello\0world"
```

You can check for it:

python

```
if "\0" in s:
```

```
    print("Contains a null byte")
```

This is a real character with ASCII value 0.

❖ 3 — Use None to represent “no string”

If you want a *true* null value (not a character):

python

```
s = None
```

But this is **not** a string. It’s Python’s null object.

❖ 4 — Insert a null via user input

If you want the user to *enter* something that you treat as null:

Empty input → empty string

python

```
value = input("Enter something: ")
```

```
if value == "":
    print("User entered null")
```

How to enter a null into a float or an integer in python

The key idea: **Python floats and integers cannot contain a “null value” inside them.** But you *can* represent “no number” in three different ways depending on what you need:

- **empty input** → treat as null
- **None** → true null value
- **NaN** → numeric null placeholder

Here’s how each works and when to use them.

✅ 1 — Use None (Python’s real null)

If you want a true null value for a number, you don’t store it *in* the float or int — you store **None** instead.

```
python
```

```
value = None
```

Example with user input:

```
python
```

```
raw = input("Enter a number: ").strip()
```

```
if raw == "":
```

```
    num = None
```

```
else:
```

```
    num = float(raw)
```

```
print("Number is:", num)
```

This is the correct way to represent “no number”.

2 — Use NaN (numeric null placeholder)

If you need a *numeric* null (common in math, pandas, NumPy), use **float('nan')**:

```
python
```

```
num = float('nan')
```

Check for it:

```
python
```

```
import math
```

```
if math.isnan(num):
```

```
    print("This is a numeric null (NaN)")
```

This is useful when you must keep the value numeric but still represent “missing”.

3 — Treat empty input as null

If the user presses Enter, `input()` returns `""`. You convert that to `None` or `NaN`:

Convert empty input → None

```
python
```

```
raw = input("Enter a number: ").strip()
```

```
num = float(raw) if raw else None
```

Convert empty input → NaN

```
python
```

```
raw = input("Enter a number: ").strip()
```

```
num = float(raw) if raw else float('nan')
```